

System Design — MCP Security Gateway

A policy-enforcement proxy for AI-agent tool calls. Companies point their MCP clients (Claude Desktop/Code, custom agents) at the gateway instead of at their MCP servers; the gateway enforces allow/block/redact/rewrite/ quarantine/approval policy on every call, tracks per-session risk and taint, and ships a tamper-evident audit trail to their SIEM.

This document records every major technical decision, the alternatives considered, and why they lost. Each decision ends with a **one-line defense** — the sentence you'd say in an interview.

1. Requirements

1.1 Functional

- Transparent interception of MCP traffic — **no changes to the agent, no changes to the MCP server.**
Adoption = pointing the client at us.
- Per-tool policy with six actions: `allow`, `block`, `redact`, `rewrite`, `quarantine`, `require_approval` — plus role-based overrides.
- Argument-level constraints (an allowed tool can still be denied for *what* it's doing) and argument rewrites (force safe forms).
- PII **and secrets** detection/redaction on both directions: tool results before they reach the LLM, arguments before they leave the boundary.
- Session-state controls: taint tracking (untrusted source → block sinks), sequence rules (forbid B after A), per-session risk scoring with auto-suspend.
- Human-in-the-loop approvals (console click, later Slack), fail-closed.
- Behavioral anomaly detection (heuristic or LLM-judged), feeding risk.
- Complete audit trail: every decision, redaction, and score change, shippable to Splunk / S3 / any SIEM.
- Pluggable **connector packs** — curated policies for specific MCP servers (GitHub, Jira, Slack, filesystem) a company can install and override.
- Ops console: live feed, sessions, replay, policy view, backtesting, approvals.

1.2 Non-functional

Property	Target	Reasoning
Added latency (policy + regex path)	p50 < 5 ms, p99 < 50 ms	An agent turn is seconds (LLM) + 50 ms–5 s (upstream API). Tens of ms is invisible.

Property	Target	Reasoning
Added latency (NER redaction path)	< 300 ms on capped payload	NER is the expensive tier; we cap/chunk input size and make the tier per-tool configurable.
Throughput	100s of calls/sec per instance	See capacity estimate — agent traffic is low-QPS, large-payload.
Availability	Sidecar: N/A (per-host). Central: 99.9 % via stateless replicas	Gateway down = agents down; central mode must be HA.
Failure posture	Fail-closed on the enforcement path	A security control that fails open is decoration. Per-tool override exists for pragmatists.
Audit delivery	At-least-once, never blocks the hot path	Local disk spool absorbs SIEM outages.
Adoption effort	< 5 minutes from install to first policed call	<code>mcp-gateway wrap -- <server cmd></code> with a sane default policy.

1.3 Capacity estimation (know these numbers)

Mid-size customer: 500 engineers, ~150 daily agent users, ~150 tool calls per user per day →

- **~22 500 tool calls/day ≈ 0.26/sec average, bursts of 10–20/sec.** Even 100× this is trivial for one async Python process doing JSON pass-through; we are **I/O-bound, not CPU-bound**.
- Payloads: args usually < 1 KB; results median a few KB, p99 ~1 MB (file reads, CI logs). **Redaction cost scales with result bytes** — hence size caps and per-tool tier selection, not a global NER pass.
- Audit: ~3 events/call × 22.5 k calls ≈ **70 k events/day × ~700 B ≈ 50 MB/day**, ~18 GB/year raw, 2–3 GB/year gzipped in S3. SQLite happily indexes years of this locally; this is *not* big data, and pretending it is would add ops cost for nothing.

The punchline: **our scale problem is adoption friction and payload inspection cost, not QPS**. Every storage/transport decision below follows from that.

2. Shape of the system: modular monolith, two deployment modes

Decision: one deployable — a modular monolith with a plugin architecture — offered in two modes:

1. **Sidecar (wrap) mode** — `mcp-gateway wrap -- npx @modelcontextprotocol/server-filesystem /data`. Speaks stdio to the client, launches the real server as a subprocess. Zero infrastructure; this is the 5-minute adoption path and the dev default.
2. **Central service mode** — `mcp-gateway serve --config gateway.yaml`. One HA cluster fronting many upstream MCP servers; agents connect over Streamable HTTP with OAuth; state in Redis/Postgres; this is the enterprise mode where policy, identity, and audit are centralized.

Same binary, same pipeline, same policy files — only transports and state stores differ. That symmetry is deliberate: a company pilots with a sidecar on one laptop, then promotes the *same policy packs* to the central cluster.

Why not microservices (policy service, redaction service, audit service):

	Microservices	Modular monolith (chosen)
Hot-path latency	+1–5 ms per network hop, per call, per service	In-process function calls
Adoption	Customer must deploy/operate N services	One container / one pip install
Failure modes	Partial failures between our own components	One process; fail-closed is simple to reason about
Team fit	Suits many teams owning services	We are one small team
Scaling	Independent scaling of NER	Only NER could ever need it — solvable later with a worker pool behind the same interface

Microservices solve organizational scaling and heterogeneous load. We have neither. The one component that could ever justify extraction (NER-based redaction on GPU) hides behind a `Detector` interface, so extraction later is a config change, not a rewrite.

One-line defense: *"Enforcement is latency-sensitive and adoption-sensitive; both argue for in-process everything, with plugin seams where future extraction might happen."*

3. Language: Python 3.12+

Decision: Python for the whole product (core, console backend, CLI).

Why:

- **The PII/security-ML ecosystem is Python.** Presidio, spaCy, transformers. A redaction subsystem is our flagship feature; building it in Go means shelling out to Python anyway or settling for regex-only.
- **MCP ecosystem:** first-class official Python SDK; most reference servers are Python or TypeScript.
- **I/O-bound workload** (§1.3): the GIL is irrelevant when you're awaiting subprocess pipes and HTTP; regex is C under the hood; spaCy is Cython.
- **Velocity and auditability:** security teams (our buyers) read Python. Policy engine logic that a customer's security engineer can audit in an afternoon is a feature.

Alternatives:

Language	Pros	Cons	Why rejected
Go	Single static binary (great distribution); goroutines; the "proxy language" (Envoy-adjacent culture)	No Presidio/spaCy; weaker ML ecosystem; slower iteration; our QPS doesn't need it	Distribution advantage is real but solvable (Docker, PyInstaller); the redaction ecosystem gap is not solvable
Rust	Performance ceiling, memory safety	Dev velocity cost is severe; ecosystem gap worse than Go; hiring pool	We would be optimizing the part of the system (proxy CPU) that is nowhere near the bottleneck
TypeScript/Node	MCP's other first-class SDK; same language as many MCP servers	PII/NER tooling weak; typing discipline lower for security-critical code	Viable runner-up; loses on the redaction subsystem, which is our differentiator

Honest disadvantages of Python, and mitigations: runtime required (→ official Docker image, `pipx` for the CLI); slower cold start (~irrelevant, long-lived process); CPU-bound NER blocks the loop (→ run detectors in a thread/process pool executor); packaging pain (→ `uv` lockfiles, extras).

One-line defense: *"The bottleneck is payload inspection, and the payload -inspection ecosystem — Presidio, spaCy — is Python; everything else about the proxy is I/O-bound, where language choice barely matters."*

4. Concurrency: asyncio end-to-end

Decision: single async event loop per process; CPU-heavy detectors dispatched to a pool executor.

- The workload is *waiting*: on client stdin, upstream pipes/HTTP, approval humans, SIEM endpoints. asyncio handles thousands of concurrent waits in one process with explicit, auditable interleaving.
- **Timeouts and cancellation are first-class** (`asyncio.timeout`), which matter enormously in a fail-closed proxy: every upstream call, approval wait, and sink flush gets a budget.
- The prototype used threads + locks; it worked at demo scale but cancellation (kill a pending approval when the client disconnects) and backpressure are exactly where threads get ugly.

Why not threads: no cancellation story, lock discipline by convention, harder to reason about ordering in a security-audit review. **Why not multiprocessing:** shatters shared session state (taint, risk) for zero benefit at our QPS.

One-line defense: *"A proxy is a scheduler of waits; asyncio makes the waits, timeouts, and cancellations explicit — which in a fail-closed system is a correctness feature, not a style choice."*

5. API surfaces — three planes, three answers

The interview trap here is treating "REST vs gRPC vs WebSocket" as one question. We have **three different planes** with different constraints.

5.1 Data plane (agent ↔ gateway ↔ MCP server): JSON-RPC 2.0 over stdio and Streamable HTTP

Not our choice — the MCP spec's. Transparency is the product; we speak exactly what the client and server speak:

- **stdio** (newline-delimited JSON-RPC) for sidecar mode.
- **Streamable HTTP** (the current MCP spec transport: POST for messages, SSE for server-initiated streams, `Mcp-Session-Id` header for session affinity) for central mode. MCP's authorization spec is **OAuth 2.1**, which is precisely where Okta/Auth0 plug in (§9).

Anything else (a REST facade, gRPC) would make us a *translation* layer — breaking transparency, chasing every MCP spec revision, and re-implementing semantics. Interception, not translation.

5.2 Control plane (console, policy, sessions, backtest): REST + OpenAPI

Decision: JSON REST, served by FastAPI, with an auto-generated OpenAPI spec.

Why REST: the consumers are a browser SPA and customers' scripts/curl. Resources map cleanly (`/api/sessions`, `/api/events`, `/api/approvals/{id}`, `/api/policy/backtest`). Cacheable, debuggable, zero client toolchain. The OpenAPI spec doubles as enterprise integration documentation for free.

Alternative	Why rejected
gRPC	Browsers need grpc-web + a proxy; protobuf toolchain for consumers; its wins (high-QPS binary service-to-service, streaming contracts) don't apply to a low-QPS management API
GraphQL	One first-party client; no over/under-fetching problem to solve; adds resolver complexity and a security surface (query-depth abuse) to a <i>security product</i>

5.3 Live events (console feed) : Server-Sent Events. Approvals outbound: webhooks

Why SSE over WebSocket: the flow is strictly server→client (decisions stream to the console; approval responses are plain POSTs back). SSE gives auto-reconnect with `Last-Event-ID` resume *in the browser for free*, rides plain HTTP (corporate proxies and load balancers don't care), and is ~20 lines to serve. WebSocket buys bidirectionality we don't use, at the cost of upgrade handling and LB configuration. **Why not polling:** latency for approvals, wasted load for feeds — though the approval *waiter* inside the gateway is effectively a long-poll with a deadline, which is fine. **Webhooks** (signed, retried) are the outbound approval channel for Slack/PagerDuty later — push, because a human is waiting.

One-line defense: "The data plane is spec-dictated JSON-RPC; the management plane is classic low-QPS resource CRUD, which is REST's home turf; the event plane is one-directional fan-out, which is SSE's home turf. Three planes, three right answers."

6. Storage

6.1 Audit events: append-only JSONL spool + SQLite index (default) → PostgreSQL (central)

Decision: every event is first appended to a local JSONL spool file (crash-safe, greppable, the shipping format for SIEM sinks). An embedded **SQLite (WAL mode)** database indexes events for the console (sessions list, replay, backtest) with zero operational cost. In central mode, the index store is **PostgreSQL with JSONB** columns.

Why this two-layer shape: the *log* is the source of truth and the transport buffer (an S3/Splunk sink reads the spool — SIEM outage never loses or blocks anything); the *index* is a disposable query accelerator, rebuildable from the log. Separating them is what makes "never block the hot path, never drop an event" cheap to guarantee.

Why SQLite: single writer (one gateway process) is its sweet spot; WAL mode lets console reads run concurrently; zero setup preserves the 5-minute adoption story. **Why Postgres for central:** real concurrency across replicas, JSONB + GIN indexes for flexible event queries, one boring system that also holds approvals and session records, every enterprise already runs it.

Alternative	Why rejected
MongoDB	JSONB in Postgres gives the schema flexibility without a second database technology; our volumes (§1.3) never touch Mongo's scaling regime
Elasticsearch	Full-text event analytics is the <i>SIEM's</i> job — the customer already has Splunk; shipping a search cluster inside a security proxy competes with our own integration story
ClickHouse	Superb at billions of events; we have 70 k/day. Ops weight for nothing
Kafka as the store	Kafka is transport, not storage, and our rates don't justify a broker; the disk spool + batching sinks give at-least-once delivery with zero infra. A Kafka <i>sink</i> can exist for customers who ask

6.2 Session/risk/taint state: in-memory (sidecar) → Redis (central)

Keyed by session and principal. **Why Redis in central mode:** taint and risk must be shared across gateway replicas or an attacker just reconnects; atomic `INCRBY` for risk scores, TTLs for session expiry, pub/sub to push suspensions cluster-wide, single-digit-ms latency on the hot path. **Why not "just use Postgres":** a DB round-trip per tool call on the enforcement path for state with TTL semantics is the wrong tool; **why not Memcached:** no persistence option, no pub/sub, no atomic structures beyond counters.

6.3 Policy: files in git — never a database

Policies are code: reviewed in PRs, versioned, diffable, signed, rolled back with `git revert`. A policy table in a DB is mutable state that bypasses review — the exact anti-pattern this product exists to prevent. The gateway loads a **validated, versioned bundle** into memory and hot-reloads only on a bundle that passes schema + signature checks (§10). Runtime evaluation is dict lookups — microseconds, no store on the hot path.

6.4 Token vault (reversible redaction): SQLite/Postgres + envelope encryption

`tokenize` redaction (token ↔ value, admin can detokenize with audit) needs encrypted at-rest storage: AES-GCM data keys wrapped by a KMS key (or local master key in sidecar mode). **Why not HashiCorp Vault as a requirement:** heavy operational demand on the customer; it becomes an optional backend behind the same `Vault` interface.

One-line defense: "Source of truth is an append-only log because audit is the product; SQLite/Postgres are query accelerators; Redis holds the only truly shared mutable state (risk/taint); and policy lives in git because mutable policy outside review is the vulnerability class we sell against."

7. Web framework (control plane + HTTP transport): FastAPI on uvicorn

Why: async-native (matches §4); Pydantic request/response validation — input validation at the edge of a *security product* should be declarative and typed; OpenAPI generated for free (§5.2); dependency-injection that makes route-level authn/authz explicit and testable; enormous community.

Alternative	Why rejected
Flask	WSGI/sync heritage; validation and OpenAPI are bolt-ons; async second-class
Django (+DRF)	ORM, admin, template engine — batteries we don't need; heavyweight for an embedded console; its sweet spot is CRUD apps, not proxies
Raw Starlette/ASGI	FastAPI is Starlette plus the validation/docs layer we want; going raw re-implements that layer by hand
Litestar	Technically fine, smaller ecosystem; no differentiator worth the smaller community

Note the boundary: FastAPI serves the **control plane** and the Streamable HTTP **endpoint**, but MCP protocol semantics (JSON-RPC correlation, session management) live in our own `protocol/` package — the framework never bleeds into enforcement logic.

8. Dependency philosophy: curated core + optional extras

The prototype's zero-dependency rule was a demo constraint. The product picks **boring, auditable, pinned** dependencies — but keeps the supply-chain surface deliberately small, because a security gateway is a high-value target:

- Core: `pydantic`, `pyyaml`, `httpx`, `anyio` — small, ubiquitous, audited.
- Extras: `mcp-gateway[server]` (FastAPI/uvicorn), `[presidio]`, `[s3]` (boto3), `[splunk]`, `[oidc]` (jwt/jwks), `[postgres]`, `[redis]`.
- The **sidecar core must run with core deps only** — regex+secrets redaction, file audit, stdio transport. A laptop pilot installs nothing heavy.

Tooling: `uv` (lockfile, fast installs), `ruff`, `pyright`, `pytest`. Distribution: PyPI + `pipx` for the CLI, an official Docker image (slim base), `docker-compose.yaml` for gateway+console quickstart, Helm chart when central mode lands.

One-line defense: *"Supply chain is attack surface for a security proxy, so the core dependency set is small and pinned, and everything heavy is an opt-in extra behind an interface."*

9. Identity & authorization

9.1 Authentication: OIDC (Okta/Auth0/Entra) primary, API keys for headless agents, static identity for sidecar

- **Central mode:** agents present a JWT bearer token; the gateway validates signature (JWKS fetch + cache), issuer, audience, expiry. This aligns with MCP's own OAuth 2.1 authorization spec — Okta/Auth0 is the authorization server, the gateway is the resource server. JWKS unreachable → serve from cache until key expiry, then **fail closed**.
- **API keys** (hashed at rest, scoped, revocable) for service/bot agents that can't do OIDC flows.
- **Sidecar mode:** stdio has no headers; identity is pinned at launch (`--role` / `--principal`) — honest about the physical limits of the transport.

Alternative	Why rejected
SAML	Browser-SSO-era XML; poor machine-to-machine story; both Okta and Auth0 speak OIDC natively
mTLS only	Strong machine identity but no user/group semantics for role mapping; offered as an <i>additional</i> layer, not the identity system
Roll-our-own tokens	Never. Key rotation, revocation, and federation are exactly what IdPs are for

9.2 Authorization: RBAC from IdP groups, with constraints as attribute checks

IdP groups → gateway roles via a declarative mapping file (`okta group "eng-oncall" → role "developer"`). Roles select policy overlays. **Why RBAC first:** it's what enterprises can reason about and audit; the per-call **constraint** system already gives attribute-level control (branch names, JQL scopes, repo allowlists) where it matters — light ABAC without the policy-language learning curve.

Why not OPA/Rego (or Cedar) as the policy engine: our action vocabulary — `redact`, `rewrite`, `quarantine`, `require_approval` — is *transformational*, not allow/deny; forcing it through a general-purpose decision engine buries the product's differentiator in a foreign language and makes policies harder for customers to read, not easier. OPA can appear later as a *constraint plugin* for customers who already standardize on Rego.

One-line defense: *"OIDC because both target IdPs and the MCP spec itself converge on OAuth 2.1; RBAC-plus-constraints because roles are what auditors audit, and our richer actions don't fit a pure allow/deny engine like OPA."*

10. Policy-as-code pipeline (GitHub CI/CD)

Policies live in a git repo. The CLI is the engine everywhere — locally, in CI, in the gateway:

- `mcp-gateway policy validate` — JSON Schema + semantic checks (unknown tools vs connector inventory, unreachable rules, invalid regexes).
- `mcp-gateway policy test` — `policy_tests.yaml`: given call → expected decision. Golden tests for policy.
- `mcp-gateway policy backtest --audit <log>` — replay history against the candidate; report newly-blocked / newly-allowed / changed-approval calls.
- GitHub Action on PR runs all three and **posts the backtest diff as a PR comment** — the reviewer approves a policy change with evidence of its blast radius, not a diff of YAML.
- On merge: CI builds a **bundle** (content hash, version, signature — sigstore/cosign or a KMS key). Gateways poll or receive a webhook, verify the signature, validate, then atomically swap — and **keep last-known-good** on any failure.

This reuses the backtester the prototype's dashboard already proved out, promoted from a UI feature to the change-management control.

11. Redaction engine placement: in-process library, tiered detectors

Runs in-process (§2). Detector tiers per redaction *profile*, chosen per tool rule: `secrets-only` (always-on regex + entropy + checksums — Luhn, SSN area rules), `standard` (+ PII regex), `strict` (+ Presidio NER, size-capped and chunked, executed in a pool executor with a latency budget). Overlapping detections merge by

confidence. Detector crash or budget overrun on the response path → **fail closed for that payload** (quarantine the result rather than release unscanned data), configurable per tool.

Why not a redaction microservice: +RTT on every payload, another thing for customers to deploy, and our volumes don't need independent scaling. The `Detector` interface is the extraction seam if GPU NER ever demands it.

12. Reliability & failure matrix

Fail-closed is the default posture on the enforcement path; every failure below has a defined behavior and an audit event:

Failure	Behavior
Policy bundle invalid on reload	Keep last-known-good, alarm
Policy invalid at startup	Refuse to start
Upstream server crash	JSON-RPC error to client; supervised restart with backoff; sessions marked
Approval channel down / timeout	Deny (<code>fail-closed</code>), audit <code>approval_unavailable</code>
Redaction detector error / over budget	Quarantine that result (never release unscanned data)
Audit SIEM sink down	Disk spool absorbs; ship on recovery; alarm on spool watermark — hot path never blocks, events never drop
Redis down (central)	Configurable: fail-closed (default) or degraded per-instance state with alarm
IdP JWKS unreachable	Cached keys until expiry, then fail closed
Gateway crash	Client sees dead server (safe direction); supervisor restarts; sessions resume via <code>Mcp-Session-Id</code>

Central-mode HA: gateway replicas are **stateless** (state in Redis/Postgres) behind a plain L7 LB — no sticky sessions needed, horizontal scaling is "add replicas."

13. Observability

- **Structured JSON logs** (the audit stream *is* the primary log; operational logs are separate and never contain payload data).
- **Prometheus metrics** (pull model — standard in every enterprise, no credentials pushed outward): decision counts by action, pipeline stage latency histograms, redaction counts by entity, approval latency, spool depth, upstream health.

- **OpenTelemetry traces** optional: one span per pipeline stage per call — gold for "why was this call slow/blocked."
- `/healthz` (liveness) and `/readyz` (policy loaded, upstream reachable).

14. Security of the gateway itself

The gateway sees everything, so it is the crown jewel:

- Least privilege: no payload persistence by default (audit stores decisions)
- redaction *counts*, raw args only where policy opts in — the prototype's backtester already handles that "partial confidence" honestly).
- Secrets (API keys, vault master keys) via env/KMS, never in config files.
- Signed policy bundles (§10); pinned, minimal dependencies (§8); SBOM published per release.
- The console requires authn even read-only — an audit feed is a reconnaissance goldmine.
- Rate limits on the control plane; the data plane inherits upstream limits.

15. Decision crib sheet

Decision	Choice	Runner-up	The one sentence
Shape	Modular monolith, plugin seams	Microservices	Latency + adoption friction both punish hops; plugin interfaces are the future-proofing
Language	Python 3.12	Go	The redaction ecosystem is Python; everything else is I/O-bound
Concurrency	asyncio	Threads	Timeouts/cancellation are correctness features in a fail-closed proxy
Data plane	MCP JSON-RPC (stdio + Streamable HTTP)	—	Spec-dictated; we intercept, we don't translate
Control plane	REST + OpenAPI (FastAPI)	gRPC	Low-QPS resource CRUD for browsers and curl is REST's home turf
Live events	SSE	WebSocket	One-directional fan-out with free reconnect; approvals are just POSTs
Audit store	JSONL spool + SQLite → Postgres JSONB	Mongo/ES/ClickHouse	The log is the truth and the SIEM buffer; indexes are disposable; SIEM does analytics
Session state	Memory → Redis	Postgres	TTL + atomic INCR + pub/sub at hot-path latency
Policy store	Git files, signed bundles	Database	Mutable policy outside review is the vulnerability class we sell against

Decision	Choice	Runner-up	The one sentence
Identity	OIDC (Okta/Auth0) + API keys	SAML	OAuth 2.1 is where the IdPs and the MCP spec itself converge
AuthZ	RBAC + per-call constraints	OPA/ABAC	Our actions transform calls, they don't just allow/deny
Framework	FastAPI	Flask/Django	Async + typed validation + free OpenAPI, no unused batteries
Redaction	In-process, tiered detectors	Sidecar service	RTT per payload is the enemy; the Detector interface is the extraction seam